Unit II – Linear Data Structures Using Sequential Organization

Prof. Pranjal Pandit

Unit II – Linear Data Structures Using Sequential Organization

- Concept of sequential organization;
- Arrays as an ADT and array operations;
- merging of two arrays; storage representation and address calculation (row-major, column-major);
- multi-dimensional arrays.
- Ordered lists;
- Single-variable polynomial representation using arrays, polynomial as array of structures, polynomial addition and multiplication.
- Sparse matrices: representation, addition, simple and fast transpose, time–space trade-off.

Arrays

An array is a collection of items of the same variable type that are stored at contiguous memory locations. It is one of the most popular and simple data structures used in programming.

Basic terminologies of Array

- **Array Element:** Elements are items stored in an array.
- **Array Index:** Elements are accessed by their indexes. Indexes in most of the programming languages start from 0.

Memory representation of Array

In an array, all the elements or their references are stored in contiguous memory locations. This allows for efficient access and manipulation of elements.

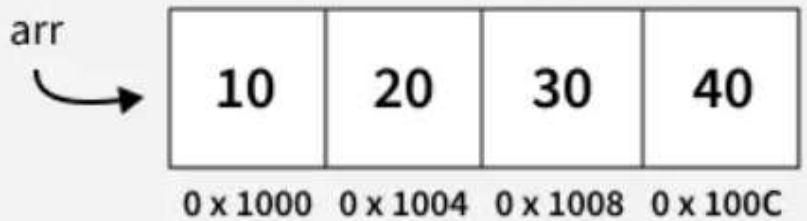
```
int arr[] = {11, 9, 17, 89, 1, 90, 19, 5, 3, 23, 43, 99}
```



In C++, arrays are stored in contiguous memory, meaning that each element is placed right next to the previous one in memory.

Primitive Array

int []arr = {10, 20, 30, 40}



arr [0] → 10

Primitive arrays stores the values directly in the memory

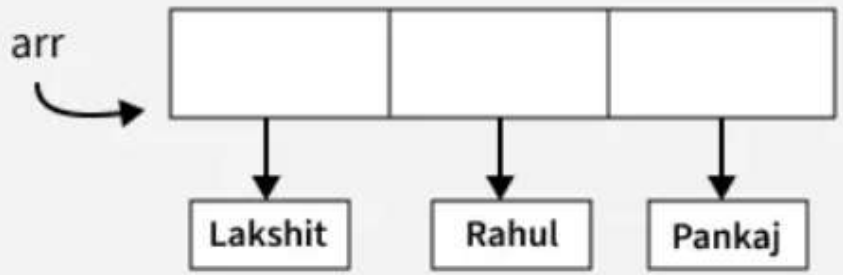
Object Array

String []arr = new String [3]

arr [0] = “Lakshit”

arr [1] = “Rahul”

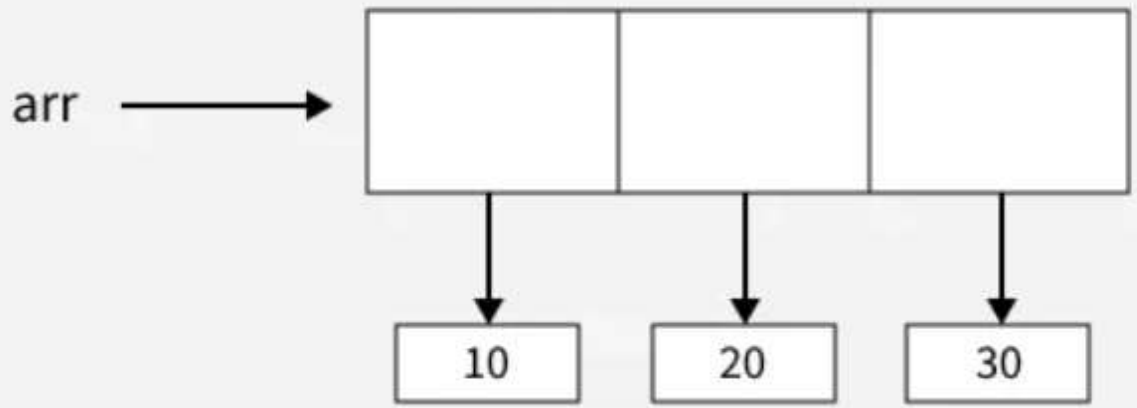
arr [2] = “Pankaj”



Each element of the object array stores a reference to separate string object

Array in Java

arr = [10, 20, 30]



In Python, lists store references to objects, meaning each element in the list points to a memory location where the object is stored.

Array in Python

Declaration of Array

Arrays can be declared in various ways in different languages. For better illustration, below are some language-specific array declarations:

C++/C

```
// This array will store integer type element
int arr[5];

// This array will store char type element
char arr[10];

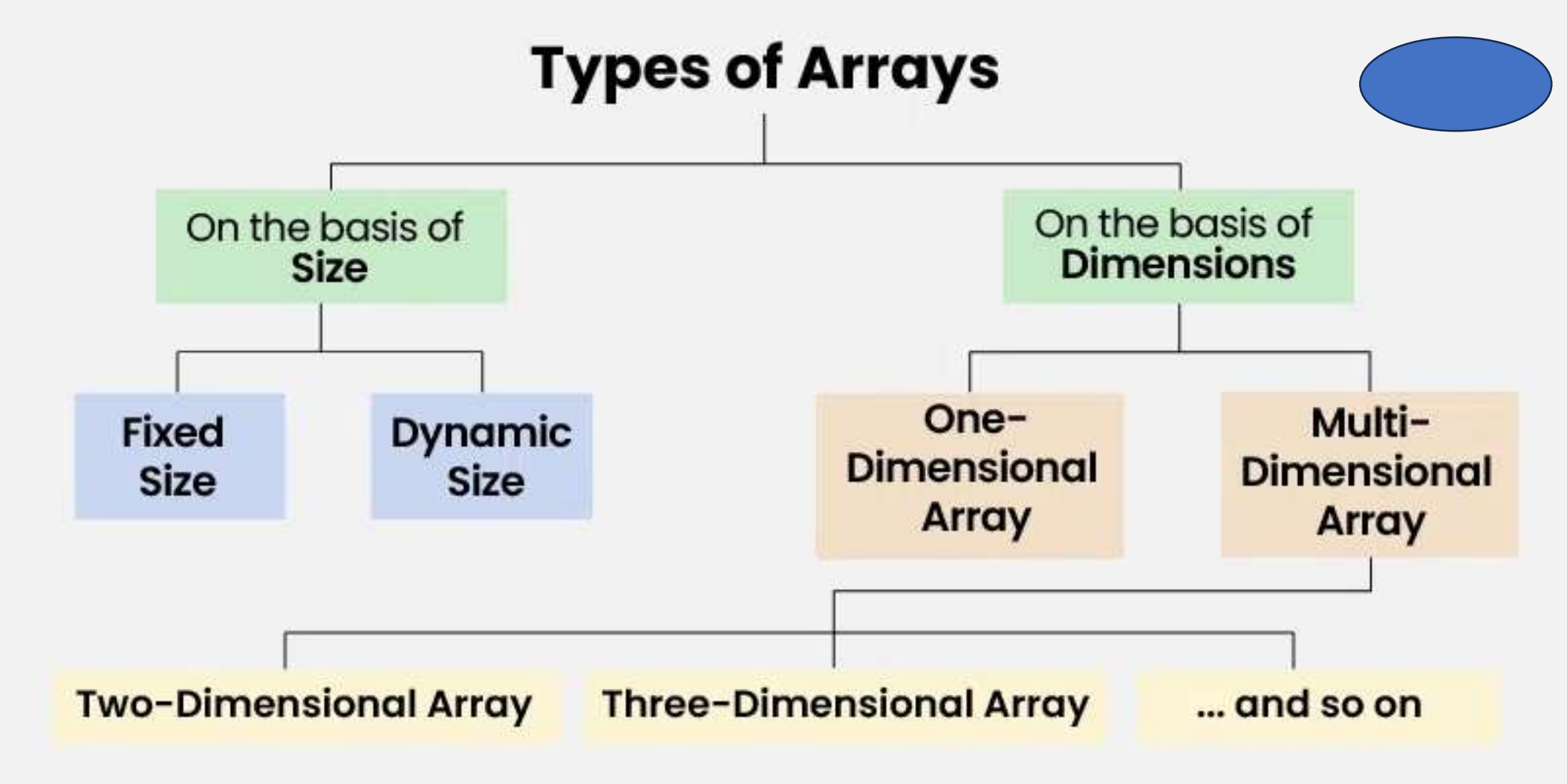
// This array will store float type element
float arr[20];
```

Initialization of Array

Arrays can be initialized in different ways in different languages. Below are some language-specific array initialization:

C/C++

```
int arr[] = { 1, 2, 3, 4, 5 };
char arr[5] = { 'a', 'b', 'c', 'd', 'e' };
float arr[10] = { 1.4, 2.0, 24, 5.0, 0.0 };
```


Types of Arrays
On the basis of Size
On the basis of Dimensions
Fixed Size
Dynamic Size
One-Dimensional Array
Multi-Dimensional Array
Two-Dimensional Array
Three-Dimensional Array
... and so on

Types of Arrays on the basis of Size

1. Fixed Sized Arrays

- We cannot alter or update the size of this array.
- Here only a fixed size (i,e. the size that is mentioned in square brackets **[]**) of memory will be allocated for storage.
- In case, we don't know the size of the array then if we declare a larger size and store a lesser number of elements, it will result in a wastage of memory.
- And if we declare a lesser size than the number of elements then we won't get enough memory to store all the elements.

C++

// Method 1 to create a fixed sized array.

// Here the memory is allocated at compile time.

```
int arr[5];
```

// Another way (creation and initialization both)

```
int arr2[5] = {1, 2, 3, 4, 5};
```

// Method 2 to create a fixed sized array

// Here memory is allocated at run time (Also known as dynamically allocated arrays)

```
int *arr = new int[5];
```

```
int n;
cin >> n;

int *arr = new int[n];   // Runtime size

delete[] arr;  // array exists until you delete it explicitly
```

Modern C++ recommendation

Although new[] is valid C++, modern C++ generally prefers using std::vector, which automatically manages memory:

```
#include <vector>
vector<int> arr(5);
```

OR

```
int n;
cin >> n;

vector<int> arr(n);
```

Advantages:

- No need for delete[]
- Automatically resizes if needed
- Safer and less error-prone
- Preferred in professional C++ code

Declaration	**Valid?**	**Memory**	**Need to free?**
`int arr[5];`	Yes	Stack (typically)	No
`int *arr = new int[5];`	Yes	Heap	`delete[] arr;`
`vector<int> arr(5);`	Yes	Managed dynamically	No (automatic)

C

// Method 1 to create a fixed sized array.
// Here the memory is allocated at compile time.

```
int arr1[5];
```

// Another way (creation and initialization both)

```
int arr2[5] = {1, 2, 3, 4, 5};
```

// Method 2 to create a fixed sized array
// Here memory is allocated at run time (Also known as dynamically allocated arrays)

```
int *arr = (int*)malloc(n * sizeof(int));
```

Java

```
// Fixed sized array examples
                                    int[] arr1 = new int [5];

// Another way (Array creation and initialization both)
                                    int[] arr2 = {1, 2, 3, 4, 5};
```

2. Dynamic Sized Arrays

- The size of the array changes as per user requirements during execution of code so the coders do not have to worry about sizes.
- They can add and removed the elements as per the need.
- The memory is mostly dynamically allocated and de-allocated in these arrays.

C++

```
#include<vector>

// Dynamic Integer Array
vector<int> v;

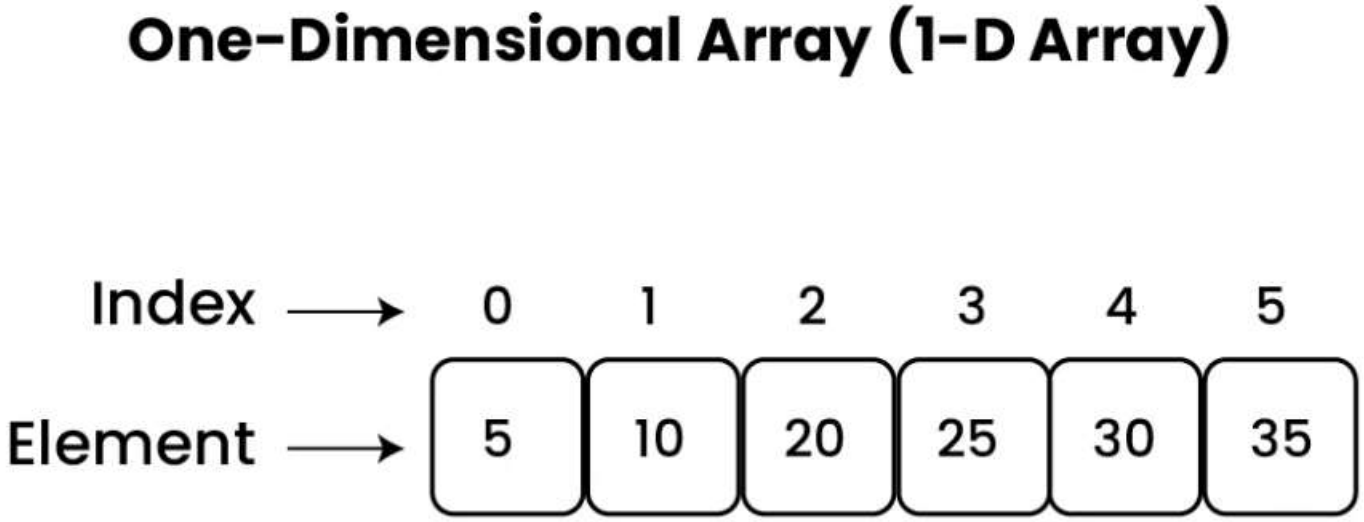
// C does not seem to support dynamic sized arrays as of now

// Dynamic Integer Array (Java)
ArrayList<Integer> arr = new ArrayList<>();

# Dynamic Array (Python)
arr = []
```

Types of Arrays on the basis of Dimensions

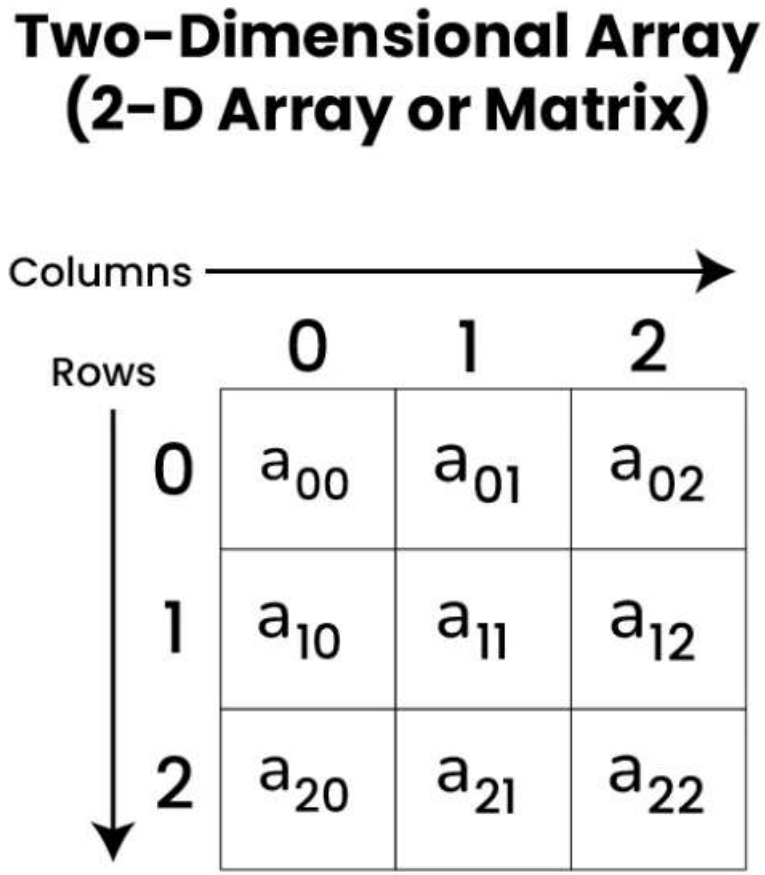
1. One-dimensional Array(1-D Array): You can imagine a 1d array as a row, where elements are stored one after another.



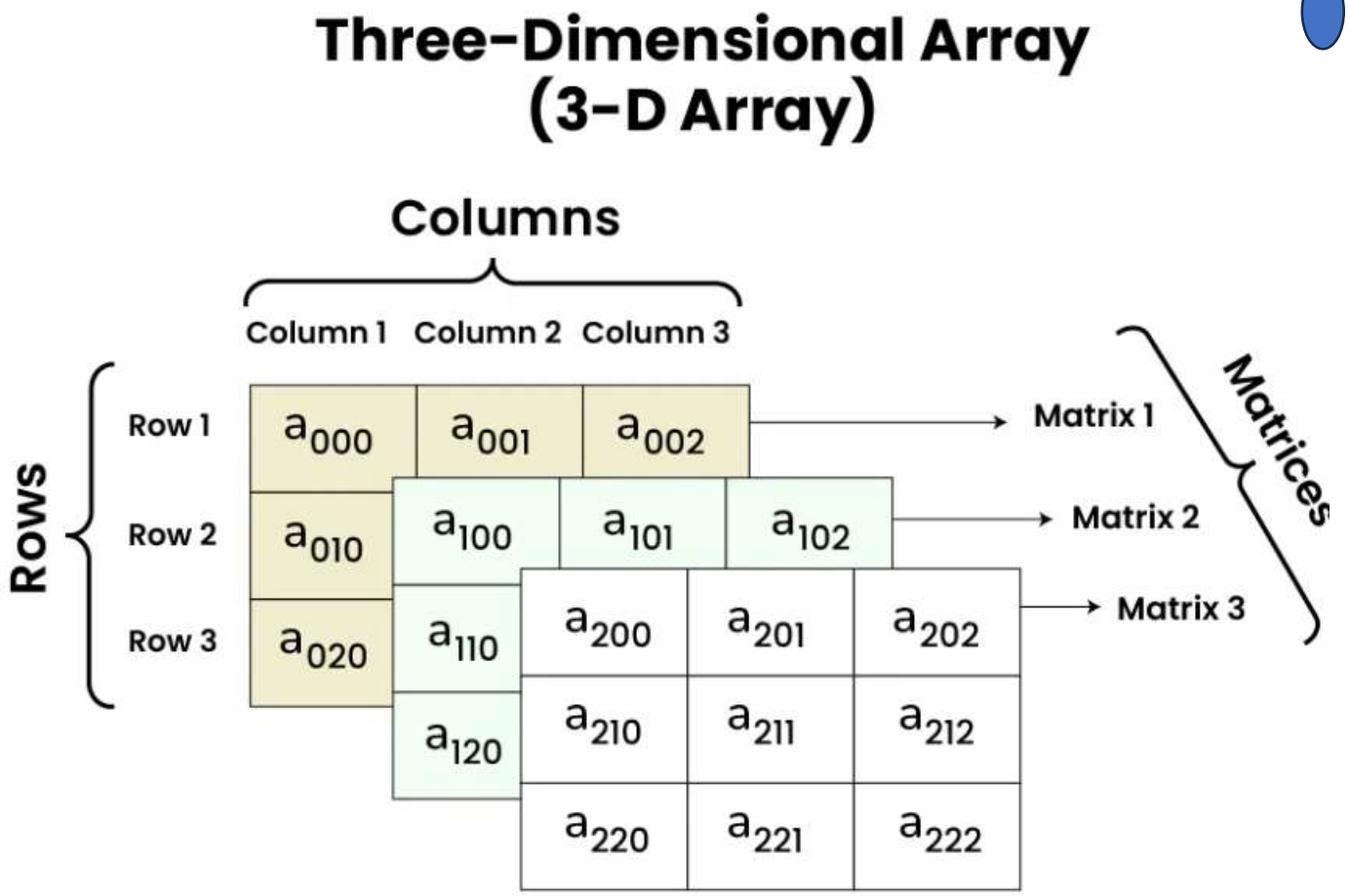
2. Multi-dimensional Array: A multi-dimensional array is an array with more than one dimension.

We can use multidimensional array to store complex data in the form of tables, etc. We can have 2-D arrays, 3-D arrays, 4-D arrays and so on.

Two-Dimensional Array(2-D Array or Matrix): 2-D Multidimensional arrays can be considered as an array of arrays or as a matrix consisting of rows and columns.



Three-Dimensional Array(3-D Array): A 3-D Multidimensional array contains three dimensions, so it can be considered an array of two-dimensional arrays.



Operations on Array

- Merging of two arrays;
- Storage representation and address calculation (row-major, column-major);
- Multi-dimensional arrays.

Merging of Two Arrays

Definition:
Merging means combining two arrays into a single array.

Example 1 (Simple Merge)

Suppose we have two arrays:

Array A = [10, 20, 30]

Array B = [40, 50, 60]

After merging:

Array C = [10, 20, 30, 40, 50, 60]

Merging 2 arrays in 3rd array

```
int A[] = {10,20,30};
int B[] = {40,50,60};
int C[6];

for(int i=0;i<3;i++)
    C[i]=A[i];

for(int i=0;i<3;i++)
    C[i+3]=B[i];
```

Example 2 (Merging Sorted Arrays)

Suppose

$$A = [2,5,8]$$

$$B = [1,3,9]$$

Merged sorted array:

$$C = [1,2,3,5,8,9]$$

Here we compare elements one by one and insert the smaller element first.

(Write the code…)

```
#include <iostream>
using namespace std;

int main()
{
    int A[100], B[100], C[200];
    int n1, n2;

    cout << "Enter number of elements in first array: ";
    cin >> n1;

    cout << "Enter elements of first sorted array:\n";
    for (int i = 0; i < n1; i++)
        cin >> A[i];

    cout << "Enter number of elements in second array: ";
    cin >> n2;

    cout << "Enter elements of second sorted array:\n";
    for (int i = 0; i < n2; i++)
        cin >> B[i];

    int i = 0, j = 0, k = 0;
```

```
// Merge the arrays
while (i < n1 && j < n2)
   {
      if (A[i] < B[j])
      {
         C[k] = A[i];
         i++;
      }
      else
      {
         C[k] = B[j];
         j++;
      }
      k++;
   }

// Copy remaining elements of A
   while (i < n1)
   {
      C[k] = A[i];
      i++;
      k++;
   }

   // Copy remaining elements of B
   while (j < n2)
   {
      C[k] = B[j];
      j++;
      k++;
   }

   cout << "\nMerged Sorted Array:\n";
   for (int x = 0; x < k; x++)
      cout << C[x] << " ";

   return 0;
}
```

Storage Representation of Arrays

An array stores elements in **contiguous memory locations**.

Example:

A = [10,20,30,40]

Assume:

Base address = 1000

Integer size = 4 bytes

Memory representation:

Address	Value
1000	10
1004	20
1008	30
1012	40

Address Calculation (1-D Array)

Formula:

$$\text{Address}(A[i]) = \text{Base Address} + i \times \text{Size of each element}$$

Suppose

A = [10,20,30,40,50]

Base Address = 2000

Size = 4 bytes

Find address of A[3].

Solution:

Address = 2000 + (3 × 4)

= 2012

So,

A[3] is stored at address 2012.

Two-Dimensional Array

A 2D array is like a table. Example:

A =

1 2 3

4 5 6

7 8 9

Rows = 3 Columns = 3

Index representation:

Col

0 1 2

Row

0 1 2 3

1 4 5 6

2 7 8 9

Example:

A[1][2] = 6

Row-Major Order

In row-major order, **all elements of the first row are stored first**, then the second row, and so on.

Example:

1 2 3

4 5 6

7 8 9

Memory layout:

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

Memory addresses (Base = 1000, Size = 4):

Address	Value
1000	1
1004	2
1008	3
1012	4
1016	5
1020	6
1024	7
1028	8
1032	9

Although it appears as rows and columns, the compiler stores the elements in **row-major order** in C.

Row-Major Address Formula

$$\text{Address}(A[i][j]) = \text{Base} + \big((i \times \text{No. of Columns}) + j\big) \times w$$

where:

Base = starting address

w= size of each element

i= row number

j= column number

Example

Matrix:

1 2 3

4 5 6

7 8 9

Find address of A[2][1]

Given:

Base = 1000
Columns = 3
Size = 4 bytes

Calculation:

```
Address = 1000 + ((2×3)+1)×4

        =1000 + (6+1)×4

        =1000 + 28

        =1028
```

At address **1028**, value = **8**.

Column-Major Order

In column-major order, **all elements of the first column are stored first**, then the second column, and so on.

Same matrix:

1 2 3

4 5 6

7 8 9

Memory layout:

1 → 4 → 7 → 2 → 5 → 8 → 3 → 6 → 9

Address	Value
1000	1
1004	4
1008	7
1012	2
1016	5
1020	8
1024	3
1028	6
1032	9

Column-Major Address Formula

$$\text{Address}(A[i][j]) = \text{Base} + \big((j \times \text{No. of Rows}) + i\big) \times w$$

Example

Find address of A[2][1]

Given:

Base = 1000

Rows = 3

Size = 4 bytes

Calculation:

Address =1000 + ((1×3)+2)×4

=1000 + (3+2)×4

=1000 +20

=1020

At address **1020**, value = **8**.

Three-Dimensional Array (3 D)

A 3D array can be thought of as multiple 2D arrays stacked together.

Example:

```
int cube[2][3][4];
```

Meaning:

2 layers

Each layer has 3 rows

Each row has 4 columns

Layer 0

1 2 3 4

5 6 7 8

9 10 11 12

Layer 1

13 14 15 16

17 18 19 **20**

21 22 23 24

cube[2][3][4];

Access:

cube[1][2][3]

This means:

Layer 1

Row 2

Column 3

Applications of Multi-Dimensional Arrays

- Matrices in mathematics
- Image processing (pixels arranged in rows and columns)
- Game boards (e.g., chess, tic-tac-toe)
- Spreadsheets
- Seating arrangements
- Scientific data and simulations

Advantages

- Organizes data in rows, columns, or higher dimensions.
- Makes it easy to represent tables and matrices.
- Simplifies processing of structured data using nested loops.

Disadvantages

- Memory usage increases as dimensions grow.
- Traversing higher-dimensional arrays requires multiple nested loops.
- Fixed size (in standard arrays) once declared.